

A dark blue vertical bar on the left side of the page. A horizontal blue arrow points to the right, overlapping the vertical bar. The date '08 /06/ 2026' is written in white inside the arrow.

08 /06/ 2026

RAPPORT PROJET INVEXTX

Several thin, curved lines in dark blue and light grey originate from the bottom left corner and curve upwards and to the right.

Présenté par :

Ahmed Rachid Bangoura
&
Aissatou Lamarana Diallo

Table des matières

1. Introduction Générale & Vision Startup-----	2
1.1 Contexte FinTech et Concept de "Paper Trading" -----	2
1.2 Enjeux Techniques d'Ingénierie Financière-----	2
1.3 Choix Technologiques-----	2
2. Sécurité & Authentification Avancée-----	4
2.1 Persistance de Session (Double Jeton JWT & Refresh Token) -----	4
2.2 Cryptographie Avancée (Hachage Renforcé Salt & Pepper) -----	4
2.4 Gestion des Profils et Téléversements Sécurisés-----	5
3. Modélisation de la Base de Données SQL & Prisma -----	6
3.1 Architecture 3-Tiers du Système -----	6
3.2 Diagramme Relationnel des Données-----	7
3.3 Schéma Prisma Détaillé -----	8
3.4 Contraintes d'Intégrité et Optimisation par Indexation -----	9
4. Logique Métier & Moteur Financier Backend -----	10
4.1 Rigueur Financière et Atomicité SQL (ACID) -----	10
4.2 Algorithme de Calcul du Prix Unitaire Moyen Pondéré (PUMP)-----	10
4.3 Dashboard Analytique et Indicateurs de Performance (P/L)-----	11
4.4 Ingestion de Données et Alertes-----	11
5. Développement Avancé A : Moteur de Matching (Order Book)-----	12
5.1 Principes de Fonctionnement et Priorité FIFO (Prix/Temps) -----	12
5.2 Exécutions Maker/Taker et Traitement des Statuts -----	12
5.3 Sécurité Transactionnelle : Anti-Double Spending -----	13
6. Développement Avancé B : Analyse de Sentiment NLP Locale -----	14
6.1 Extraction de Flux Finnhub et Détection Bilingue de Langue -----	14
6.2 Pipeline de NLP Mathématique Local "From Scratch" -----	14
6.3 Scoring Lexical et Classification Finale -----	15
7. Conclusion Générale et Perspectives -----	16

1. Introduction Générale & Vision Startup

1.1 Contexte FinTech et Concept de "Paper Trading"

Le secteur technologique de la finance (FinTech) connaît une mutation rapide avec l'émergence d'outils d'investissement grand public. Toutefois, l'apprentissage de l'investissement sur les marchés boursiers est souvent freiné par les risques de perte de capital réel.

InvestX répond à cette problématique en proposant une plateforme web de simulation boursière réaliste, reposant sur le concept de Paper Trading.

À la création de son compte, chaque utilisateur bénéficie d'une allocation initiale fictive de 100 000 \$ (USD), lui permettant de s'initier aux dynamiques de marché sans aucun risque financier.

L'environnement simule fidèlement le comportement d'une véritable banque de courtage :

- **Intégration de données réelles** : Récupération des cours boursiers réels en temps réel.
- **Typologie d'ordres réalistes** : Support des ordres d'achat et de vente immédiats (Market) et différés (Limit).
- **Précision financière** : Traitement sans déviation des calculs de solde, des gains et des pertes au centime près.

1.2 Enjeux Techniques d'Ingénierie Financière

Contrairement à une application CRUD classique, InvestX a été architecturée selon des normes d'ingénierie financière de niveau bancaire :

Cohérence Transactionnelle (ACID) : Impossibilité absolue de scinder une transaction

Concurrence Réduite : Protection contre les attaques de double dépense d'actifs fictifs ou les soldes négatifs lors de requêtes simultanées.

Sécurité Cryptographique et Résilience : Hachage de mot de passe à l'épreuve des collisions, sessions robustes et contrôle d'accès basé sur les rôles.

Data Science : Intégration locale de modèles sémantiques (Natural Language Processing) pour aider les utilisateurs à évaluer le sentiment des actualités financières sans dépendre d'APIs tierces lentes et coûteuses.

1.3 Choix Technologiques

Afin d'assurer robustesse, maintenabilité, les choix techniques suivants ont été retenus :

- **Frontend** : React pour une interface utilisateur (SPA) interactive, fluide, réactive et optimisée pour l'analyse visuelle.
- **Backend** : Node.js associé au Framework Express.js, entièrement codé en Type Script pour garantir la sécurité du typage lors des opérations financières complexes.
- **ORM** : Prisma Client pour gérer les modèles de données relationnels de manière typée et assurer des migrations fluides.

- **Base de Données :** PostgreSQL pour stocker de façon structurée et relationnelle les entités, en tirant profit de ses performances transactionnelles élevées et de ses fonctionnalités de verrouillage SQL.
- **Flux Financiers :** Intégration de l'API Finnhub pour consommer les cotations boursières mondiales, les historiques et les dépêches d'actualités d'entreprises.

2. Sécurité & Authentification Avancée

2.1 Persistance de Session (Double Jeton JWT & Refresh Token)

La persistance de session sur InvestX allie sécurité renforcée et fluidité de navigation via l'usage de jetons JSON Web Tokens (JWT) séparés :

- **Access Token** : Contient l'identité et les rôles de l'utilisateur. Sa durée de vie est courte (15 minutes) pour minimiser l'impact en cas d'interception. Il est envoyé dans l'en-tête HTTP Authorization : Bearer <token> lors des appels API.
- **Refresh Token** : Valable 7 jours, ce jeton sert uniquement à régénérer l'Access Token lorsque celui-ci expire. Pour éviter les failles XSS (Cross-Site Scripting), il est stocké côté client dans un cookie sécurisé configuré avec les attributs :
 - o **HttpOnly** : Interdit l'accès au cookie via des scripts JavaScript.
 - o **Secure** : Force la transmission exclusive via HTTPS.
 - o **SameSite=Strict** : Empêche l'envoi du cookie lors de requêtes cross-site, neutralisant les attaques CSRF (Cross-Site Request Forgery).

Un middleware Express d'autorisation intercepte les requêtes privées, décode et valide la signature de l'Access Token. Si le token est expiré, le client React intercepte l'erreur **401** et appelle de manière transparente l'Endpoint de rafraîchissement `/api/auth/refresh` pour récupérer un nouveau jeton sans déconnecter l'utilisateur.

2.2 Cryptographie Avancée (Hachage Renforcé Salt & Pepper)

L'algorithme de hachage standard `bcrypt` est le standard de l'industrie, mais il présente une limite technique historique : il tronque les mots de passe de plus de 72 octets.

Pour outrepasser cette limite et blinder le stockage des mots de passe contre les attaques par dictionnaire ou tables de correspondance (Rainbow Tables), InvestX implémente un système de hachage renforcé combinant Salt (propre à bcrypt) et Pepper (clé globale secrète) :

1. Génération du Pepper : Une clé secrète fixe et robuste (Pepper) est configurée dans les variables d'environnement du serveur.
2. Hachage HMAC : Avant de transmettre le mot de passe à bcrypt, le backend calcule un code d'authentification de message cryptographique (HMAC-SHA256) combinant le mot de passe utilisateur et le Pepper secret.
3. Bcrypted Hash : La chaîne hexadécimale de 64 caractères résultante (256 bits) est ensuite passée à `bcrypt` avec un coût de calcul (Salt rounds) de 12.

Mot de passe brut —> HMAC-SHA256 (avec Pepper secret) —> Chaîne de 64 octets —> Bcrypt (Salt=12) —> Hash Final BDD

Cette architecture protège la base de données : "USER" et "ADMIN".

- **Middlewares d'autorisation** : Des middlewares ciblés filtrent les requêtes selon les rôles décodés du JWT.
- **Panneau d'Administration** : Un ensemble d'endpoints API d'administration permet :

- **La supervision globale** : Consultation des statistiques de liquidité de la plateforme (total cash, total assets).
- **La gestion des comptes** : Bannissement, promotion de modérateurs ou suppression d'utilisateurs.
- **Suppression en Cascade** : Les tables SQL de PostgreSQL sont liées avec la directive onDelete : Cascade (définie dans Prisma).
La suppression d'un utilisateur supprime automatiquement et proprement son portefeuille ("Wallet "), ses positions ("PortfolioItem"), ses transactions ("Transaction") et ses alertes ("Alert"), évitant les enregistrements orphelins.

2.4 Gestion des Profils et Téléversements Sécurisés

Les utilisateurs disposent d'un ensemble de routes pour modifier leur profil et changer leur mot de passe.

- **Téléversement d'Avatar** : Géré par le middleware multer.
Le backend sécurise le téléversement en vérifiant le type MIME réel (uniquement les images de type PNG/JPG sont autorisées) et en rejetant tout fichier dépassant 2 Mo pour prévenir les attaques par déni de service (Dos).

3. Modélisation de la Base de Données SQL & Prisma

3.1 Architecture 3-Tiers du Système

L'application adopte une architecture découplée de type RESTful visant la robustesse, la sécurité et la haute performance pour du traitement de données financières

🚦 Niveau Présentation (Frontend React)

- **Interface React SPA & Visualisation de Données** : L'interface utilisateur est développée sous forme d'Application Web à Page Unique (SPA) avec React.

Elle intègre la librairie *Recharts* pour générer des graphiques financiers avancés en chandeliers (OHLC : *Open-High-Low-Close*), essentiels pour représenter visuellement l'évolution des actifs (PortfolioAsset) et l'historique du marché.
- **Client Axios & Intercepteurs** : Les communications HTTP vers le backend sont assurées par Axios. Des intercepteurs sécurisés sont configurés pour injecter automatiquement le jeton d'authentification (JWT) dans les en-têtes de chaque requête.

🚦 Niveau Application (Backend Node.js & Express)

Ce niveau concentre l'intelligence applicative et la sécurité :

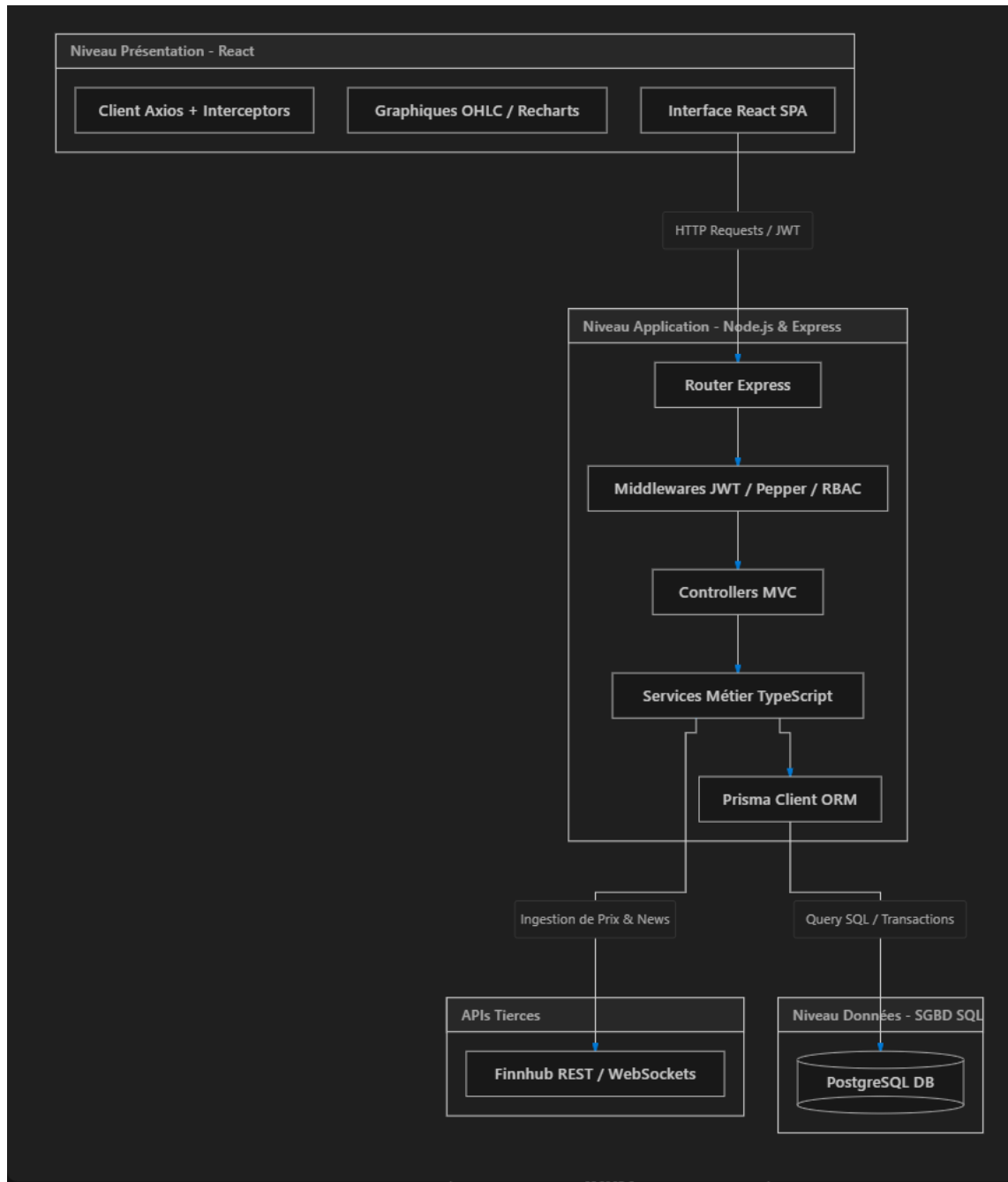
- **Routeurs Express & Contrôleurs MVC** : L'arborescence du code respecte le paradigme Modèle-Vue-Contrôleur, facilitant la maintenance et le routage des endpoints de l'API.
- **Middlewares de Sécurité (JWT, Pepper & RBAC)** :
 - *Authentication* : Vérification des jetons JWT et validation cryptographique (Salt & Pepper) liés au modèle User.
 - *RBAC (Role-Based Access Control)* : Contrôle d'accès strict basé sur la hiérarchie des rôles (USER, ADMIN) définis dans le schéma de la base de données.
- **Services Métier (TypeScript)** : C'est le cœur du système. Cette couche isole la logique complexe, comme l'algorithme d'exécution du carnet d'ordres ou le déclenchement asynchrone des alertes de prix (PriceAlert).
- **Prisma Client (ORM)** : Assure la liaison directe et typée entre la logique backend et la base de données via le fichier `schema.prisma`.

🚦 Intégration de Flux Externes (API Tierces)

- **Finnhub (REST & WebSockets)** : Cette intégration constitue la clé de voûte de la simulation boursière. L'API Finnhub alimente la plateforme en cotations boursières en temps réel pour l'évaluation dynamique des portefeuilles, et fournit les dépêches financières (titres et textes bruts) indispensables au pipeline local d'analyse de sentiment (NLP).

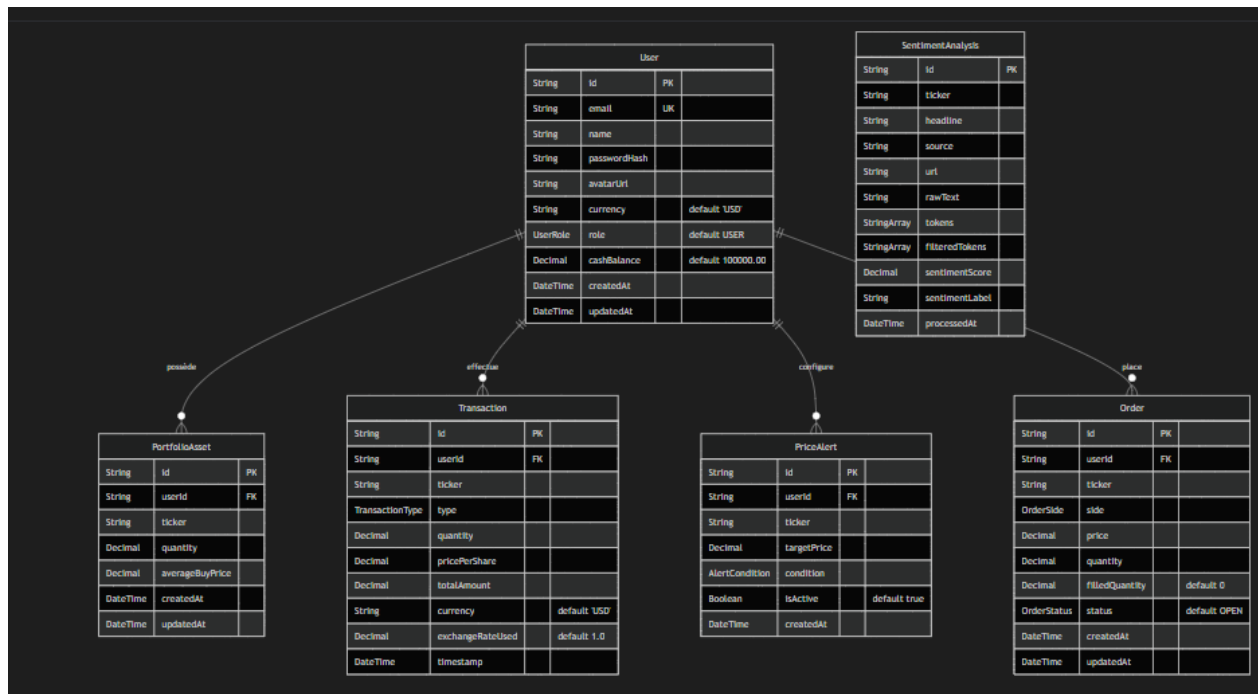
🚦 Niveau Données (Base de données)

- **PostgreSQL** : Agissant comme socle de persistance (déclaré via le provider PostgreSQL dans Prisma), cette base de données relationnelle garantit l'intégrité absolue et transactionnelle des données financières de la plateforme.



3.2 Diagramme Relationnel des Données

Le schéma ci-dessous modélise l'architecture de notre base de données PostgreSQL :



3.3 Schéma Prisma Détaillé

Le schéma de la base de données exploite les types avancés de PostgreSQL pour garantir la fiabilité financière du système :

- Précision Numérique Financière** : Afin de pallier les imprécisions de calcul inhérentes au type standard Float, les champs monétaires et quantitatifs exploitent les types natifs SQL @db. Decimal. Une précision de 18,4 (18 chiffres dont 4 décimales) est appliquée aux valeurs monétaires, et 18,6 aux volumes d'actifs, garantissant ainsi l'exactitude absolue des transactions financières au centime près.
- Intégrité Référentielle Stricte** : La cohérence de la base de données est assurée par un couplage fort entre les entités (Utilisateurs, Portefeuilles, Transactions). L'implémentation de la contrainte onDelete : Cascade assure un nettoyage rigoureux et automatique des données dépendantes lors de la suppression d'un compte utilisateur, prévenant structurellement toute persistance de données orphelines.
- Typage Fort et Sécurisation des États** : L'utilisation d'énumérations SQL (Enums) fige les différents états métier directement au niveau du moteur de base de données (ex : types d'ordres MARKET/LIMIT, statuts d'exécution). Ce mécanisme interdit l'insertion de valeurs inattendues, consolidant de fait la sécurité et la stabilité du moteur de trading face à d'éventuelles anomalies applicatives.

3.4 Contraintes d'Intégrité et Optimisation par Indexation

Pour assurer des performances constantes lors des phases de forte activité boursière, des index SQL physiques ont été implémentés au niveau de PostgreSQL :

- Index composite unique sur PortfolioItem (walletId, ticker) : Assure qu'aucun utilisateur ne possède plusieurs lignes distinctes pour le même actif, et accélère la recherche des actifs lors des passages d'ordres.
- Index composite ordonné sur Transaction (walletId, executedAt DESC) : Optimise le temps de réponse lors du chargement des journaux de transactions (tri par date décroissante pour l'utilisateur).
- Contrainte de solde positif (CHECK Constraint) : Pour garantir qu'aucun wallet ne passe en solde négatif (solvabilité d'urgence), la base de données PostgreSQL est configurée avec une contrainte SQL native :

```
ALTER TABLE "Wallet" ADD CONSTRAINT "wallet_cash_balance_positive" CHECK  
("cashBalance" >= 0.0000)
```

4. Logique Métier & Moteur Financier Backend

4.1 Rigueur Financière et Atomicité SQL (ACID)

L'implémentation du moteur financier d'InvestX garantit une atomicité absolue des ordres d'achat ou de vente. Il est hors de question qu'une transaction échoue à mi-chemin comme débiter le compte mais ne pas pouvoir allouer l'action par manque de connexion ou plantage du serveur.

Pour cela, le backend encapsule toutes les opérations au sein de transactions SQL managées par Prisma (prisma.Transaction), structurée en 5 étapes clés

1. **Verrouillage Pessimiste (FOR UPDATE)** : La ligne du portefeuille est instantanément verrouillée en base de données pour empêcher les requêtes simultanées et bloquer toute tentative de double dépense (*double-spending*).
2. **Validation de Solvabilité** : Le système vérifie rigoureusement que les fonds disponibles sont suffisants pour couvrir l'ordre.
3. **Mouvement de Fonds** : Le solde en cash du portefeuille est débité (ou crédité) du montant exact.
4. **Allocation et Ajustement du PUMP** : Les actifs sont transférés et le Prix Unitaire Moyen Pondéré (PUMP) est dynamiquement recalculé pour assurer un suivi précis de la performance.
5. **Journalisation** : L'opération est gravée dans un registre immuable pour assurer la traçabilité complète (audit) de la transaction.

L'atout majeur de cette méthode est son atomicité : à la moindre erreur durant l'une de ces étapes, le système effectue un *rollback* automatique, annulant intégralement la transaction pour éviter toute corruption des soldes.

4.2 Algorithme de Calcul du Prix Unitaire Moyen Pondéré (PUMP)

Le PUMP reflète le coût moyen d'acquisition par unité de stock, nécessaire pour mesurer la performance.

Formule mathématique :

$$PUMP_{nouveau} = \frac{(PUMP_{actuel} \times Q_{existante}) + (P_{nouveau} \times Q_{nouvelle})}{Q_{existante} + Q_{nouvelle}}$$

- **Ventes d'actifs** : N'affectent jamais le PUMP. Elles diminuent simplement la quantité possédée. Le gain/perte (P/L) est calculé au moment précis de la vente par :

$$\text{Profit/Perte Réalisé} = Q_{\text{vendue}} \times (P_{\text{vente}} - \text{PUMP})$$

4.3 Dashboard Analytique et Indicateurs de Performance (P/L)

L'Endpoint `/api/portfolio/dashboard` calcule en direct la performance globale de l'utilisateur :

1. Récupération de la liste des `portfolioItems` possédés par l'utilisateur.
2. Ingestion des derniers prix du marché (`lastPrice`) pour chaque ticker via le service de cache connecté à Finnhub.
3. Evaluation de l'Equity:

$$Equity = Cash_Balance + \sum_i (Quantity_i \times lastPrice_i)$$

4. Calcul du Profit/Loss (P/L) par rapport à l'allocation de départ :

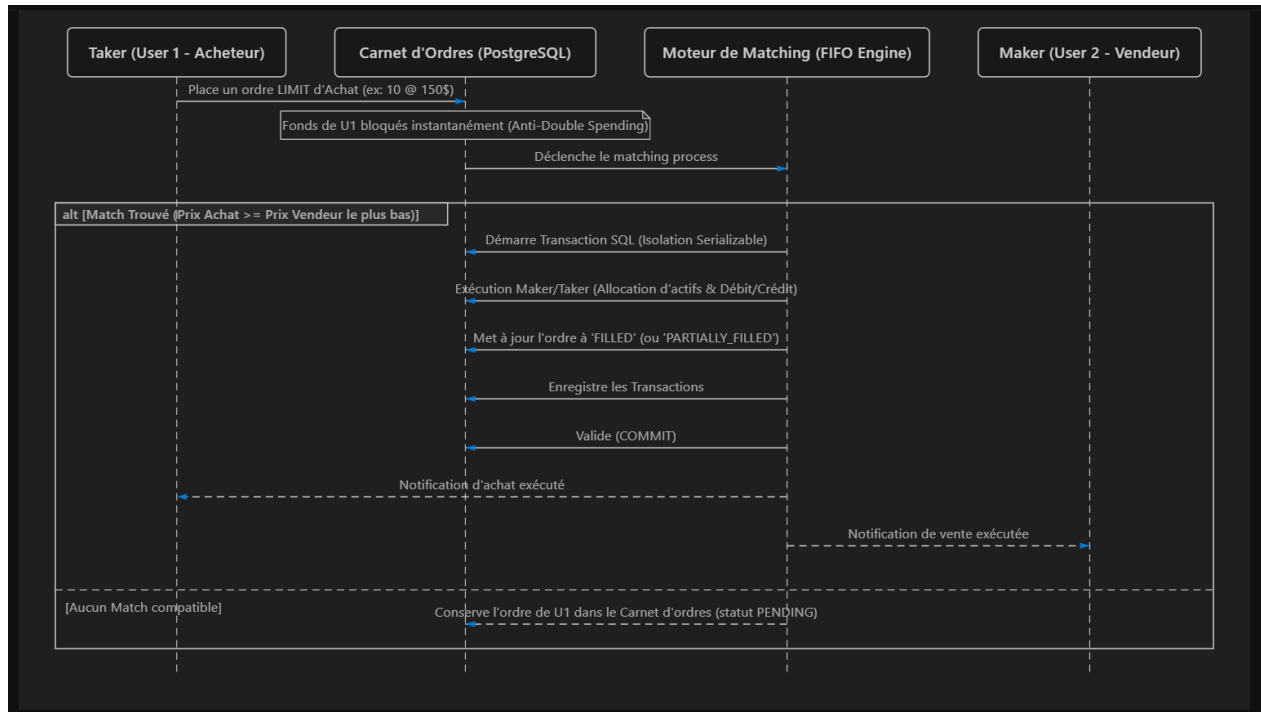
4.4 Ingestion de Données et Alertes

SOURCE FINNHUB : Une couche d'abstraction service encapsule les requêtes HTTP vers l'API Finnhub. Pour contourner les limitations strictes de requêtes (Rate Limits) de l'API publique, les prix sont stockés temporairement dans un cache local en mémoire pour une durée de 5 secondes.

MOTEUR D'ALERTE : Un worker en tâche de fond scrute régulièrement le prix des tickers pour lesquels des alertes sont configurées avec le statut PENDING. Si la condition de franchissement est validée, l'alerte passe au statut TRIGGERED et l'événement est notifié

5. Développement Avancé A : Moteur de Matching (Order Book)

Le moteur de Matching (Internal Order Book) simule une bourse fermée entre les utilisateurs d'InvestX, leur permettant d'échanger des actifs à prix fixé sans dépendre de l'API externe.



5.1 Principes de Fonctionnement et Priorité FIFO (Prix/Temps)

Le carnet d'ordres classe les ordres limites selon deux axes de priorité :

1. **Priorité de Prix** : Les ordres d'achat avec le prix le plus élevé et les ordres de vente avec le prix le plus bas sont traités en premier.
2. **Priorité de Temps** : À prix identique, le premier ordre enregistré dans le système (timestamp le plus ancien) est exécuté en premier (First-In, First-Out).

5.2 Exécutions Maker/Taker et Traitement des Statuts

Le moteur de matching gère la rencontre d'ordres aux quantités asymétriques :

FILLED (Totalemment exécuté) : L'ordre entrant correspond parfaitement en quantité à l'ordre existant.

PARTIALLY_FILLED (Partiellement exécuté) : Si la quantité de l'ordre entrant dépasse la quantité disponible sur le meilleur ordre opposé, l'ordre existant est marqué ` FILLED ` , et l'ordre entrant est exécuté pour la quantité disponible puis son reliquat reste dans le carnet avec le statut ` PARTIALLY\_FILLED ` .

Maker vs Taker : L'ordre préexistant dans le carnet d'ordres est le Maker (apporteur de liquidité). L'ordre entrant qui déclenche l'exécution est le Taker (preneur de liquidité). Le prix d'exécution final correspond systématiquement au prix fixé par le Maker.

5.3 Sécurité Transactionnelle : Anti-Double Spending

Pour empêcher qu'un utilisateur n'utilise deux fois le même argent ou le même stock en plaçant des ordres limites multiples, InvestX applique une politique stricte de Réserve Immédiate de Fonds :

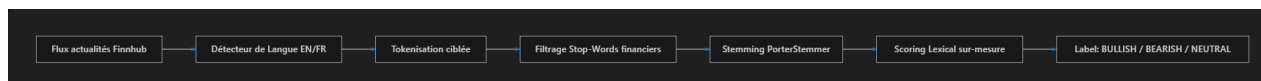
- À la pose d'un ordre LIMIT d'achat : Le montant maximum nécessaire est immédiatement soustrait du cashBalance disponible de l'utilisateur et placé dans un état "bloqué/réservé".
- À la pose d'un ordre LIMIT de vente : Les actions concernées sont immédiatement soustraites de la quantité disponible dans le portefeuille utilisateur.

En cas d'annulation de l'ordre : Les fonds ou actifs réservés sont instantanément restitués.

Cette règle élimine tout risque de double dépense en cours de Matching concurrent.

6. Développement Avancé B : Analyse de Sentiment NLP Locale

Le module d'analyse de sentiment s'appuie sur un pipeline de traitement de langage naturel (NLP) complet, hébergé et exécuté 100% localement sur le serveur backend, sans appel à des APIs externes, garantissant souveraineté, rapidité et contrôle total du modèle.



6.1 Extraction de Flux Finnhub et Détection Bilingue de Langue

1. Récupération des actualités : Le serveur interroge régulièrement l'API de news de Finnhub pour récupérer les dépêches récentes associées à un ticker boursier spécifique.
2. Détection Bilingue de Langue :

Le système prend en charge nativement l'anglais et le français. Avant de lancer le traitement sémantique, il analyse la structure lexicale du texte en comptant la probabilité de présence de Stop-Words spécifiques (ex : the, is, and pour l'anglais vs-le, est, dans, pour le français). Le texte est ensuite orienté vers le dictionnaire et le tokenizer appropriés.

6.2 Pipeline de NLP Mathématique Local "From Scratch"

Une fois la langue isolée, le texte traverse le pipeline de nettoyage et de vectorisation :

1. Tokenisation Adaptée :
 - Pour l'anglais : Utilisation d'un WordTokenizer standard découpant le texte sur les espaces et caractères spéciaux.
 - Pour le français : Utilisation de l'algorithme AggressiveTokenizerFr optimisé pour préserver la structure des mots accentués (ex. société, bénéfice).
2. Filtrage du Bruit (Stop-Words Financiers) :

Le dictionnaire supprime les mots grammaticaux vides de sens (de, et, that, with), mais filtre également le bruit de fond sémantique financier neutre n'apportant aucun sentiment décisionnel (stock, share, quarter, action, trimestre).
3. Stemming (Raciolisation) :

Application de l'algorithme PorterStemmer (ou équivalent francisé) pour ramener les mots à leur racine morphologique, regroupant ainsi les variations d'un même concept.

6.3 Scoring Lexical et Classification Finale

Le backend calcule la polarité du texte grâce à un lexique financier bilingue sur-mesure contenant des valeurs de sentiment d'intensité variable :

Lexique (Mots racines)	Valeur Affectée	Sentiment Associé
bankrupt / faillit	-5	Très Baissier
loss / pert	-2	Baissier
lawsuit / procès	-3	Baissier
Profit / bénéféc	+3	Haussier
growth / croissanc	+3	Haussier
bullish / haussi	+4	Très Haussier

Calcul du score mathématique : Le backend somme les valeurs de sentiment de tous les tokens issus de l'article : Classification finale : L'article est ensuite étiqueté selon des seuils de décision stricts $\geq +2.0$ BULLISH (Optimiste) stricts ≤ -2.0 BEARISH (Pessimiste) $-2.0 < \text{Score} < +2.0$ NEUTRAL (Neutre)

L'agrégation de ces classifications offre aux utilisateurs de la plateforme une jauge prédictive synthétisant le climat actuel de l'actif.

7. Conclusion Générale et Perspectives

Le développement de la plateforme InvestX dépasse le cadre d'une simple architecture Web CRUD traditionnelle. En intégrant des contraintes de sécurité et d'intégrité de niveau bancaire, elle pose des bases solides pour une application FinTech performante :

- **Transactions SQL ACID et verrous pessimistes** (FOR UPDATE) assurant une solvabilité sans faille.
- **Sécurité cryptographique hybride** (Salt & Pepper avec HMAC-SHA256 + bcrypt) protégeant les identifiants utilisateurs de manière optimale.
- **Moteur de Matching d'Order Book FIFO** et système de réservation de fonds, permettant d'exécuter des transactions entre utilisateurs en circuit fermé.
- **Pipeline NLP bilingue local** offrant une analyse sémantique temps réel sans appel à des APIs cloud externes.

Perspectives

L'architecture a été pensée pour être évolutive. Les futures améliorations s'axeront sur :

- **Scalabilité des ordres** : Utiliser des files d'attente pour gérer de plus grands volumes de transactions en asynchrone.
- **Amélioration de l'IA (NLP)** : Intégrer du *Machine Learning* pour affiner l'analyse de sentiment des actualités financières.
- **Social Trading** : Ajouter un système de classement (*Leaderboard*) et de partage de stratégies (*Copy Trading*) pour engager la communauté.
- **Alertes Externes** : Envoyer des notifications de prix directement par SMS ou sur mobile (Push) pour un suivi proactif.